

KMEANS

Rachit Vargas (MBA | Engineer)

rachit.vargas@ulead.ac.cr

(+506) 8474-3543

1. Introducción al clustering

1.1. Motivación

- En muchos problemas disponemos de un conjunto de **datos numéricos sin etiquetar** y queremos descubrir estructuras naturales en los datos.

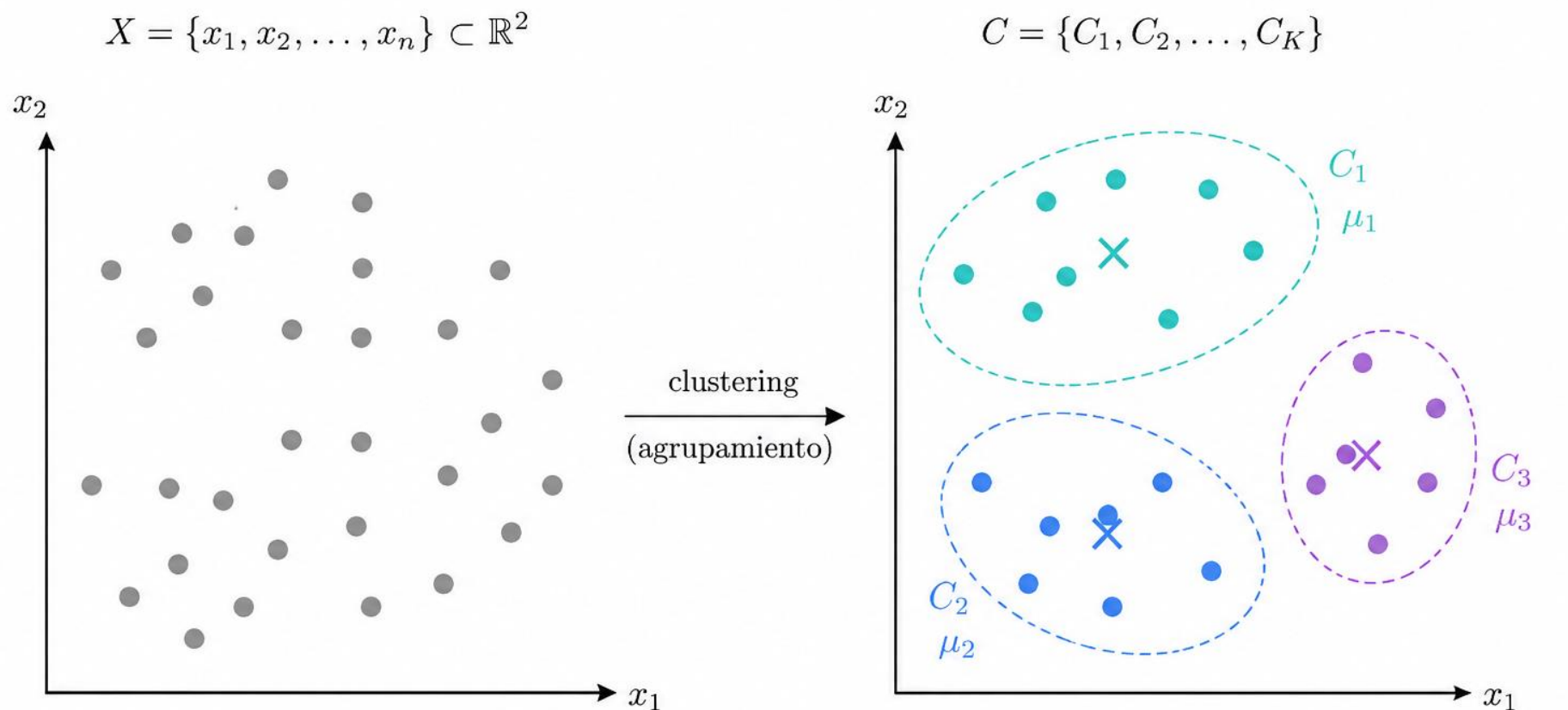
1.2. Planteamiento del problema.

Sea:

$$X = \{x_1, x_2, \dots, x_n\} \subset \mathbb{R}^p$$

- un conjunto finito de n observaciones (puntos en $\mathbb{R}^p$), donde $x_i = (x_{i1}, \dots, x_{ip})^T$ es el vector de características de la observación i .
- **Las observaciones no poseen etiquetas de clase.**
- El objetivo del *clustering* es agrupar los datos en subconjuntos (*clusters*) homogéneos internamente y heterogéneos entre sí.

1. Introducción al clustering



Datos sin etiquetar

$$C_k \subset X, C_k \neq \emptyset, C_i \cap C_j = \emptyset \ (i \neq j), \bigcup_{k=1}^K C_k = X$$

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

1. Introducción al clustering

El clustering aborda el problema de descubrir estructura en datos no etiquetados mediante la identificación de grupos naturales. Formalmente, dados datos $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^p$, se busca una partición $\mathcal{C} = \{C_1, \dots, C_K\}$ que revele agrupamientos significativos.

2.1. Descubrimiento de estructura intrínseca

Muchas bases de datos contienen observaciones generadas por mecanismos o poblaciones distintas. El clustering permite estimar esta estructura latente sin información previa:

$$X \sim \bigcup_{k=1}^K \mathcal{P}_k,$$

donde $\mathcal{P}_k$ representa la distribución asociada al grupo k .

2.2. Reducción de complejidad

Agrupar observaciones similares permite representar el conjunto de datos mediante un número mucho menor de prototipos (por ejemplo, centroides μ_k):

$$X \approx \{\mu_1, \dots, \mu_K\}, \quad \mu_k \in \mathbb{R}^p.$$

Esto facilita el almacenamiento, la visualización y el análisis posterior.

2. Creación de clusters

2.1. Una buena partición debe cumplir:

- **Alta cohesión interna:**
los puntos de un mismo clúster deben ser lo más similares posible (D_{intra}).
- **Alta separación externa:**
los clusters distintos deben ser lo más diferentes posible (D_{inter}).

$$D_{intra}(C_k) = \frac{1}{|C_k|} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Donde:

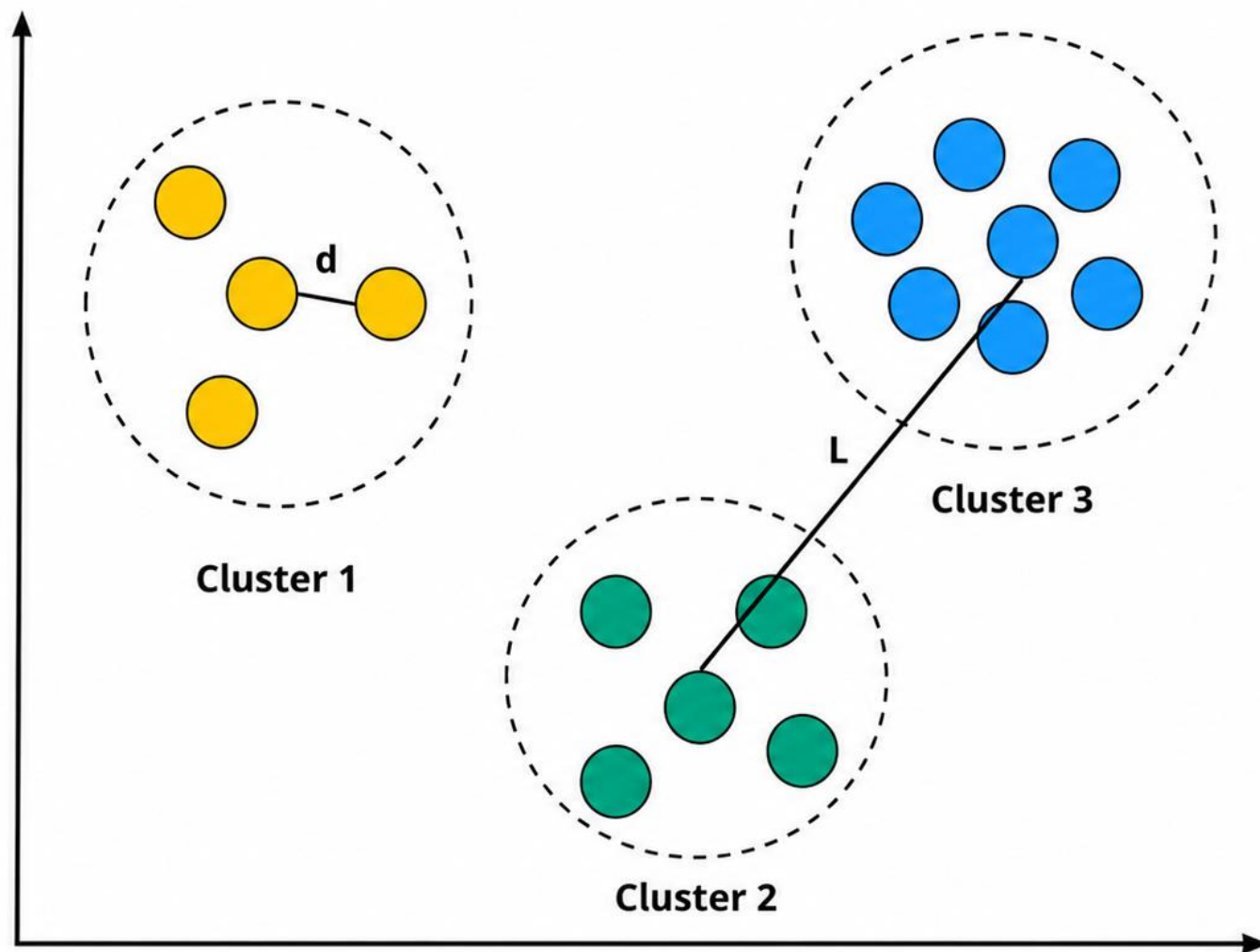
- C_k : cluster k
- $|C_k|$: número de elementos del cluster
- x_i : punto del cluster
- μ_k : centroide del cluster
- $\|x_i - \mu_k\|^2$: distancia euclidiana cuadrada

$$D_{inter}(C_i, C_j) = \|\mu_i - \mu_j\|^2$$

Donde:

- μ_i : centroide del cluster i
- μ_j : centroide del cluster j

2. Creación de clusters



d : Intra Cluster Distance

L : Inter Cluster Distance

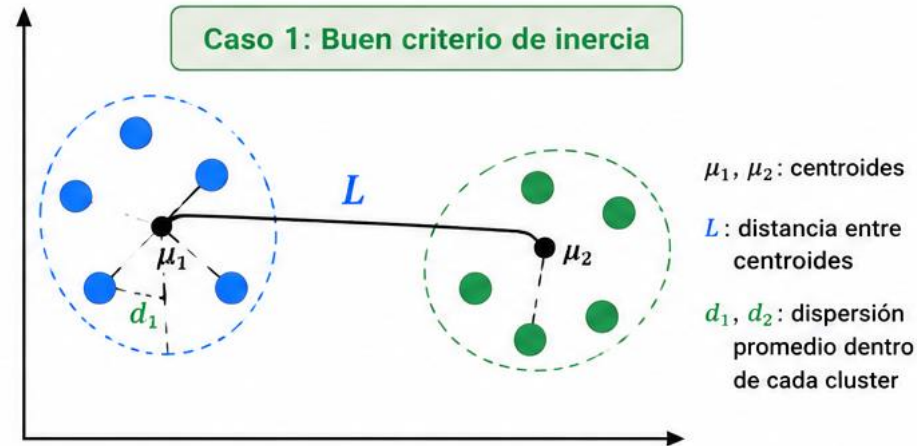
Criterio de Inercia (Teorema de Fisher)

$$J = \frac{L}{d}$$

L : distancia íter cluster (separación entre centroides)

d : distancia intra cluster (dispersión promedio dentro del cluster)

Se busca maximizar J (cluster bien separados y compactos).



Supongamos:

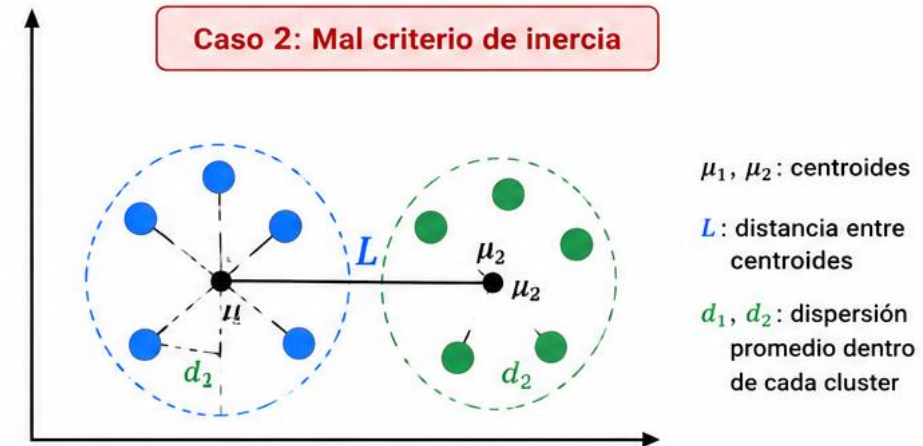
$$L = 10.0$$

$$d_1 = 0.80, d_2 = 0.70$$

$$d = \frac{d_1 + d_2}{2} = \frac{0.80 + 0.70}{2} = 0.75$$

$$J = \frac{L}{d} = \frac{10.0}{0.75} = \mathbf{13.33}$$

J es grande → Clusters bien separados y compactos



Supongamos:

$$L = 2.0$$

$$d_1 = 1.60, d_2 = 1.40$$

$$d = \frac{d_1 + d_2}{2} = \frac{1.60 + 1.40}{2} = 1.50$$

$$J = \frac{L}{d} = \frac{2.0}{1.50} = \mathbf{1.33}$$

J es pequeño → Clusters poco separados y/o poco compactos

Conclusión: Maximizar $J = \frac{L}{d}$ → Maximiza separación entre clusters (L grande) y minimiza dispersión interna (d pequeña).

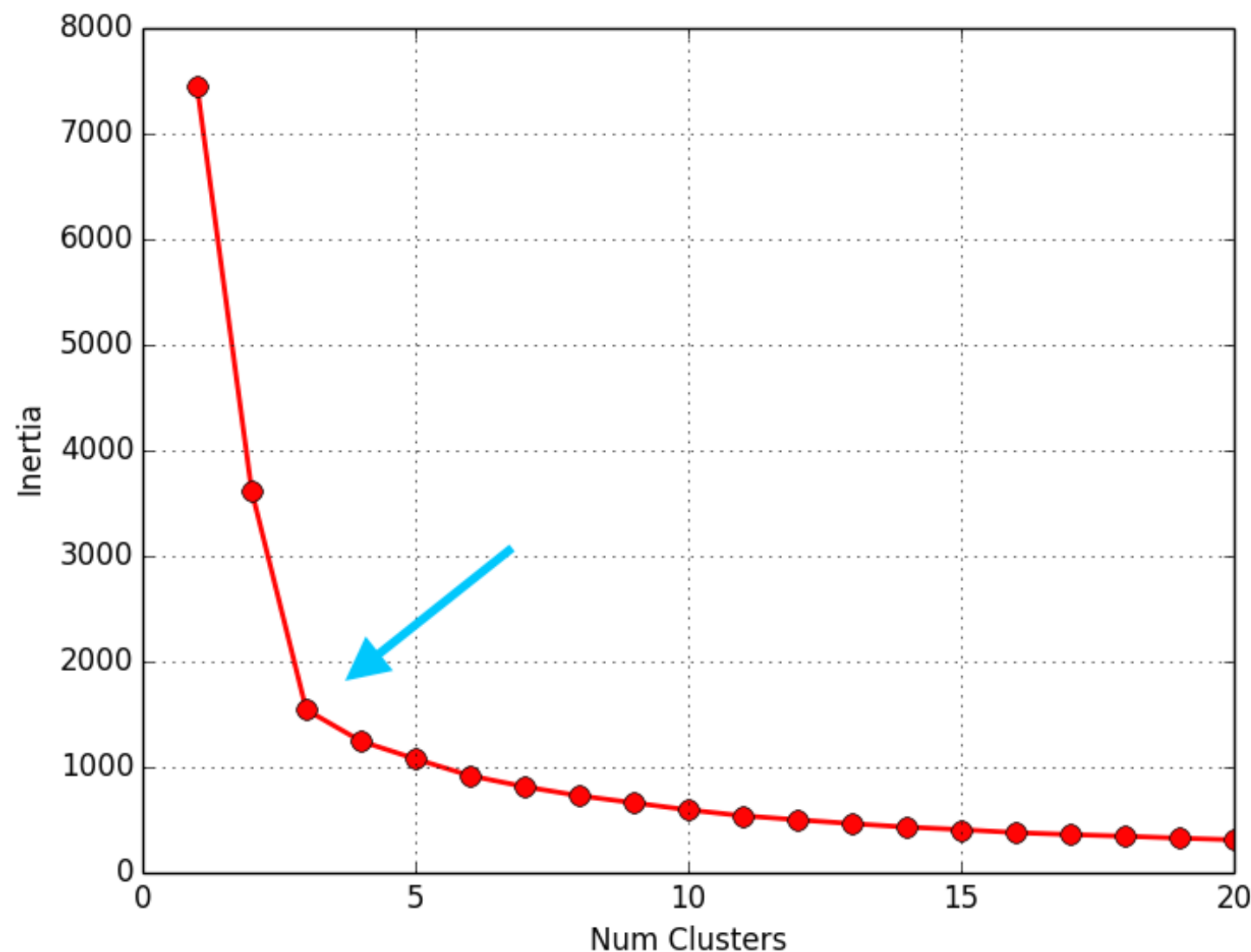
3. Codo de Jambú

- El **codo de Jambú** (más conocido como *Elbow Method* o *método del codo*) es una técnica utilizada en **clustering**, especialmente en algoritmos como **K-Means**, para decidir cuántos clusters K son adecuados.
- La idea principal es analizar cómo cambia la **inercia** cuando aumentamos el número de clusters.
- La **inercia** mide qué tan compactos son los clusters.

$$W(K) = \sum_{i=1}^K \sum_{x \in C_i} ||x - \mu_i||^2$$

- Donde:
- C_i :cluster i
- μ_i :centroide del cluster
- $||x - \mu_i||^2$:distancia cuadrática entre un punto y su centroide

3. Codo de Jambú



$K = 1$ clúster

- Todos los puntos están metidos en un solo grupo.
- los puntos quedan muy lejos del centroide
- la inercia es enorme.

$K = 2$ clúster

- Ahora el algoritmo puede separar mejor los datos.
- Resultado: la inercia baja muchísimo.

$K = 3$ clúster

- Los clusters se vuelven todavía más compactos.
- La inercia vuelve a bajar mucho.

4. Kmeans o K-medias

- **K-Means** es un algoritmo de aprendizaje no supervisado usado para clustering. Su objetivo es dividir los datos en K grupos similares. Cada grupo recibe el nombre de clúster.
- El algoritmo trabaja usando distancias entre puntos. Los puntos cercanos tienden a quedar en el mismo clúster. Los puntos lejanos suelen separarse en clusters distintos.

$$\text{Distancia Euclidiana} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- La letra K representa la cantidad de clusters deseados. Ese valor debe definirse antes de ejecutar el algoritmo. Elegir un buen K es una parte importante del proceso.
- Cada cluster posee un centro llamado centroide. El centroide representa el promedio de los puntos del grupo. Es el “centro geométrico” del cluster.

4. Kmeans o K-medias

1 Algoritmo KMEANS(X, K)

2 Inicializar centroides $\mu_1, \dots, \mu_K$ aleatoriamente de entre los puntos de X

3 Repetir

 // Paso 1: Asignación de puntos

4 Para cada punto $x \in X$ hacer

5 | Asignar x al cluster C_i cuyo centroide μ_i sea el más cercano:

6 | $i = \arg \min_{j \in \{1, \dots, K\}} \|x - \mu_j\|^2$

 // Paso 2: Recalcular centroides

7 Para cada cluster $C_i, i = 1, \dots, K$ hacer

8 | Recalcular el centroide μ_i como el promedio de los puntos en C_i :

9 |
$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

 // Paso 3: Verificar convergencia

10 Si los centroides no cambian (o cambian menos que ϵ)

11 | entonces **Detener**

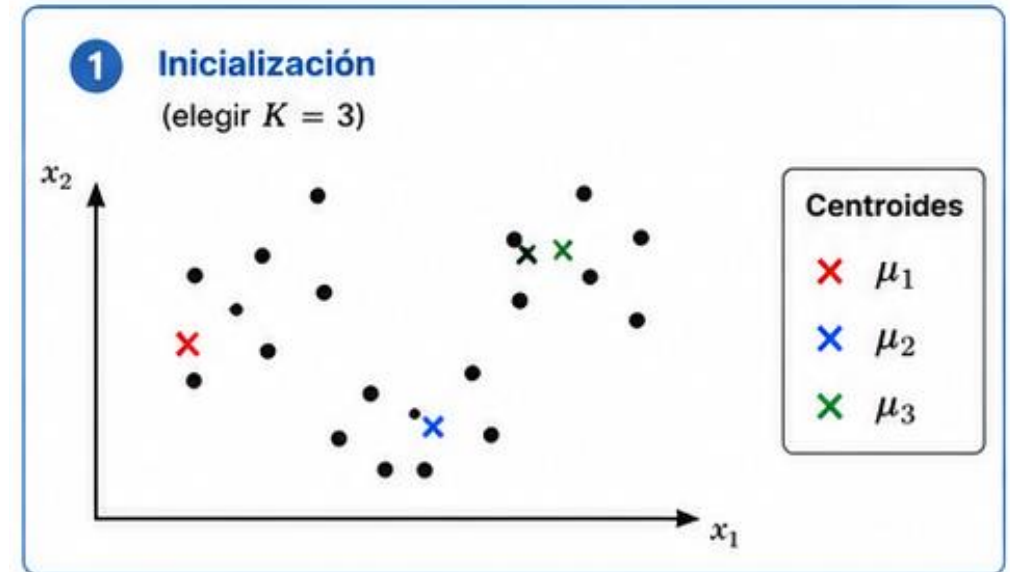
12 Sino

13 | Volver al Paso 1

14 Hasta convergencia

15 Devolver clusters $C_1, \dots, C_K$ y centroides $\mu_1, \dots, \mu_K$

16 Fin Algoritmo



4. Kmeans o K-medias

1 Algoritmo KMEANS(X, K)

2 Inicializar centroides $\mu_1, \dots, \mu_K$ aleatoriamente de entre los puntos de X

3 Repetir

 // Paso 1: Asignación de puntos

4 Para cada punto $x \in X$ hacer

5 | Asignar x al cluster C_i cuyo centroide μ_i sea el más cercano:

6 | $i = \arg \min_{j \in \{1, \dots, K\}} \|x - \mu_j\|^2$

 // Paso 2: Recalcular centroides

7 Para cada cluster $C_i, i = 1, \dots, K$ hacer

8 | Recalcular el centroide μ_i como el promedio de los puntos en C_i :

9 |
$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

 // Paso 3: Verificar convergencia

10 Si los centroides no cambian (o cambian menos que ϵ)

11 | entonces **Detener**

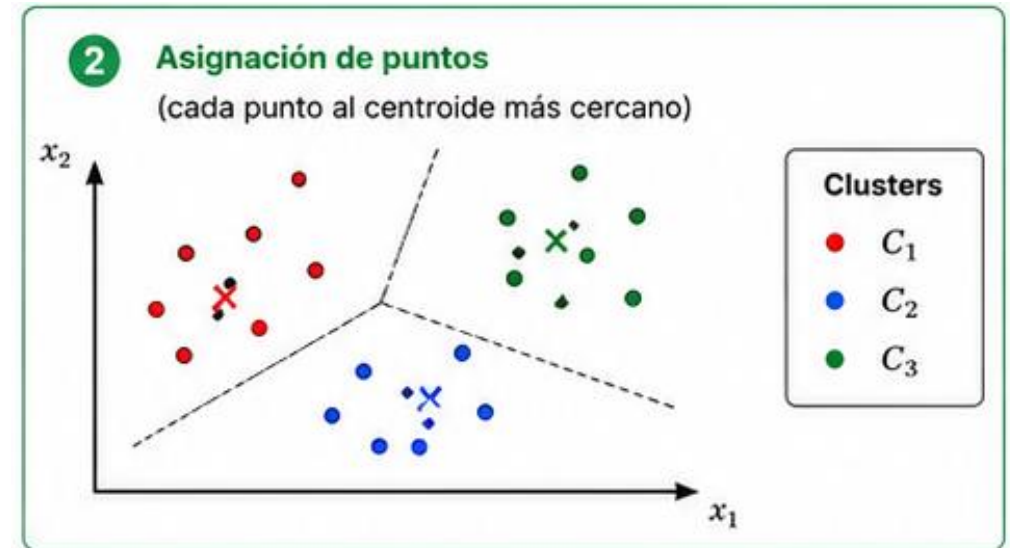
12 Sino

13 | Volver al Paso 1

14 Hasta convergencia

15 Devolver clusters $C_1, \dots, C_K$ y centroides $\mu_1, \dots, \mu_K$

16 Fin Algoritmo



4. Kmeans o K-medias

1 Algoritmo KMEANS(X, K)

2 Inicializar centroides $\mu_1, \dots, \mu_K$ aleatoriamente de entre los puntos de X

3 Repetir

// Paso 1: Asignación de puntos

4 Para cada punto $x \in X$ hacer

5 | Asignar x al cluster C_i cuyo centroide μ_i sea el más cercano:

6 | $i = \arg \min_{j \in \{1, \dots, K\}} \|x - \mu_j\|^2$

// Paso 2: Recalcular centroides

7 Para cada cluster C_i , $i = 1, \dots, K$ hacer

8 | Recalcular el centroide μ_i como el promedio de los puntos en C_i :

9 |
$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

// Paso 3: Verificar convergencia

10 Si los centroides no cambian (o cambian menos que ϵ)

11 | entonces **Detener**

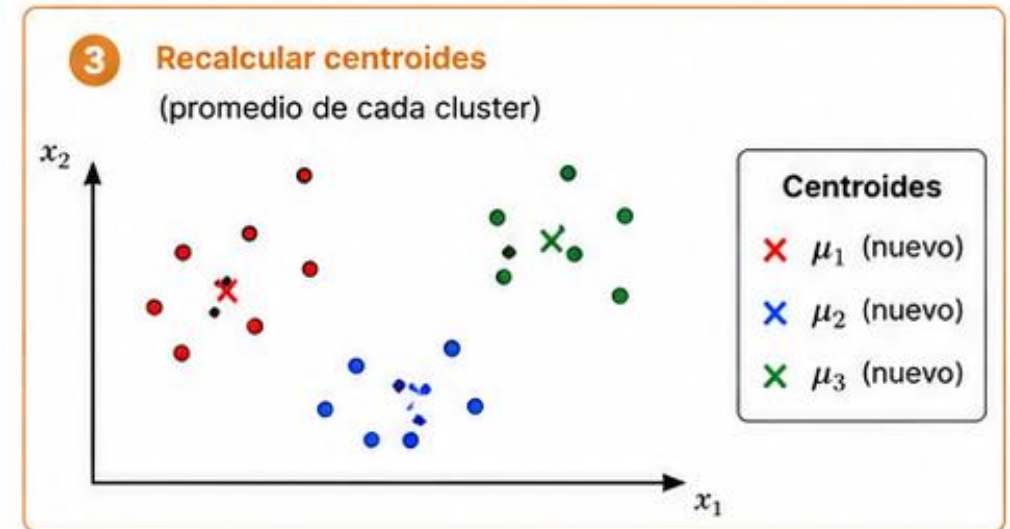
12 Sino

13 | Volver al Paso 1

14 Hasta convergencia

15 Devolver clusters $C_1, \dots, C_K$ y centroides $\mu_1, \dots, \mu_K$

16 Fin Algoritmo



4. Kmeans o K-medias

1 Algoritmo KMEANS(X, K)

2 Inicializar centroides $\mu_1, \dots, \mu_K$ aleatoriamente de entre los puntos de X

3 **Repetir**

 // Paso 1: Asignación de puntos

Para cada punto $x \in X$ **hacer**

 Asignar x al cluster C_i cuyo centroide μ_i sea el más cercano:

$$i = \arg \min_{j \in \{1, \dots, K\}} \|x - \mu_j\|^2$$

 // Paso 2: Recalcular centroides

Para cada cluster $C_i, i = 1, \dots, K$ **hacer**

 Recalcular el centroide μ_i como el promedio de los puntos en C_i :

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

 // Paso 3: Verificar convergencia

 Si los centroides no cambian (o cambian menos que ϵ)

 entonces **Detener**

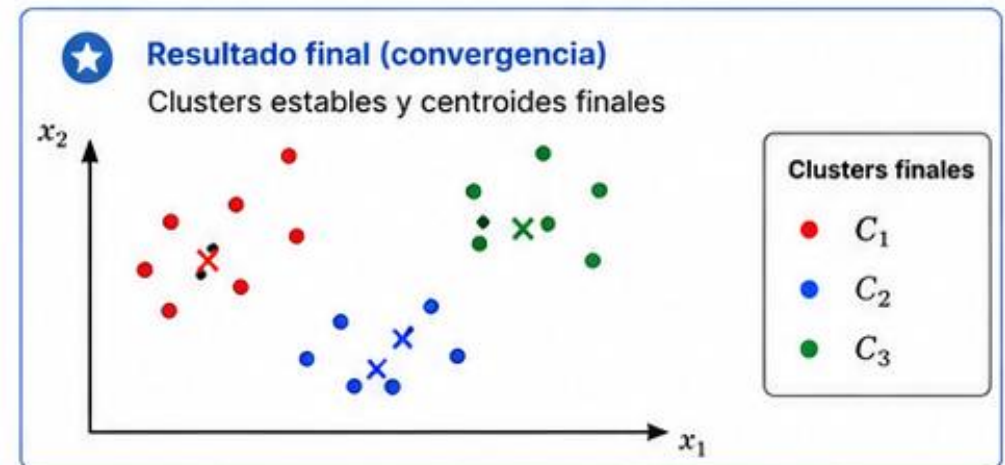
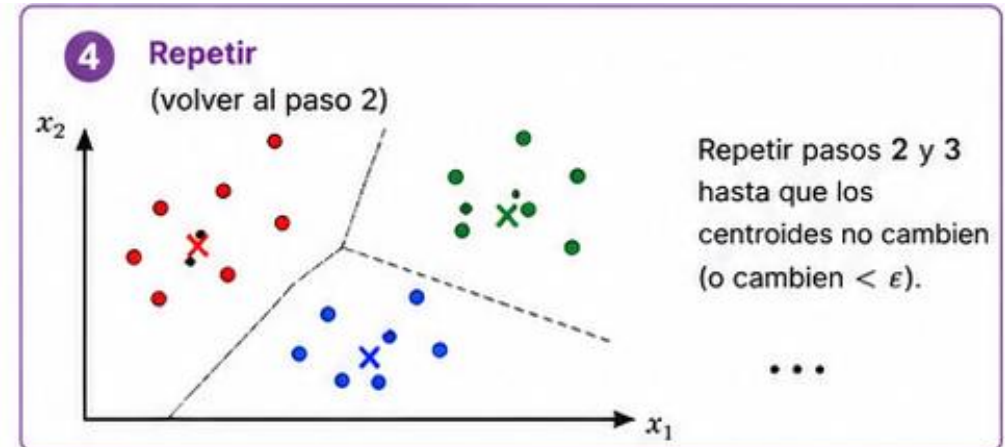
Sino

 Volver al Paso 1

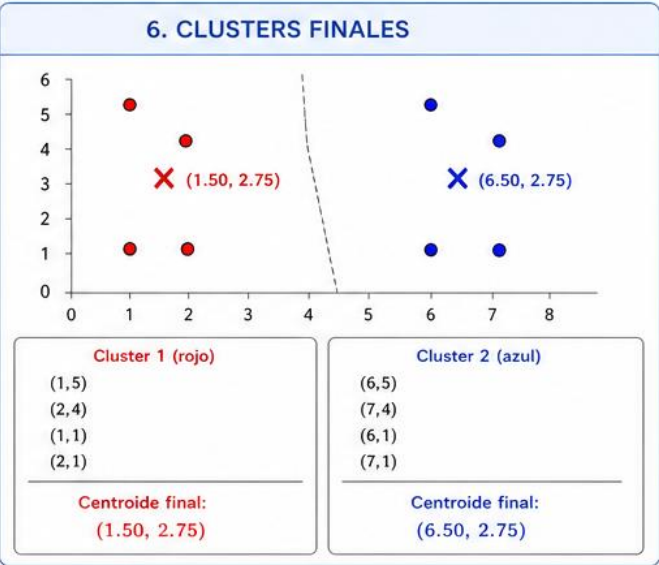
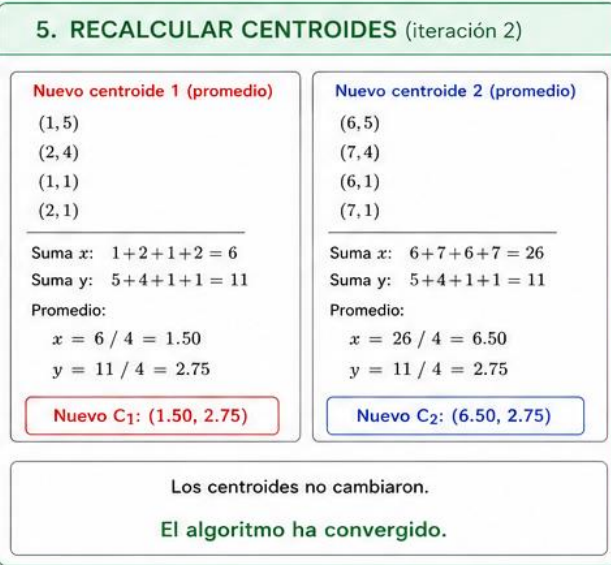
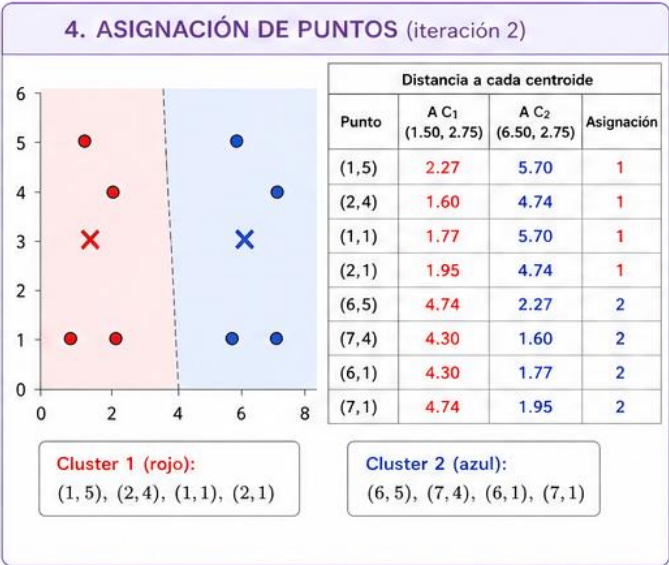
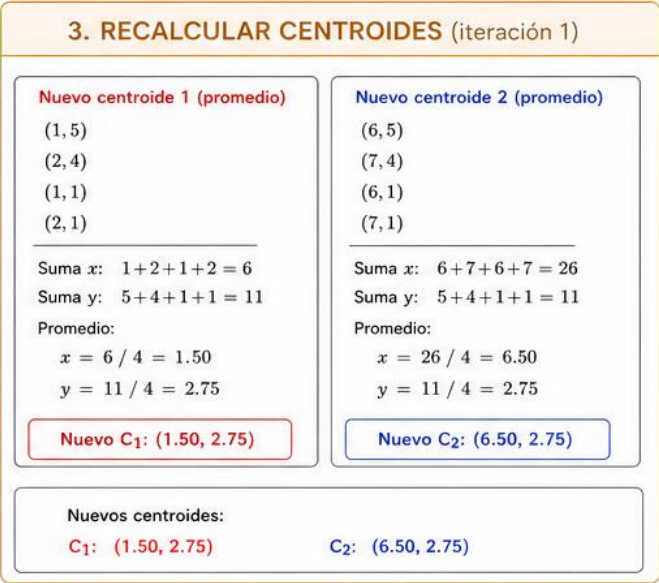
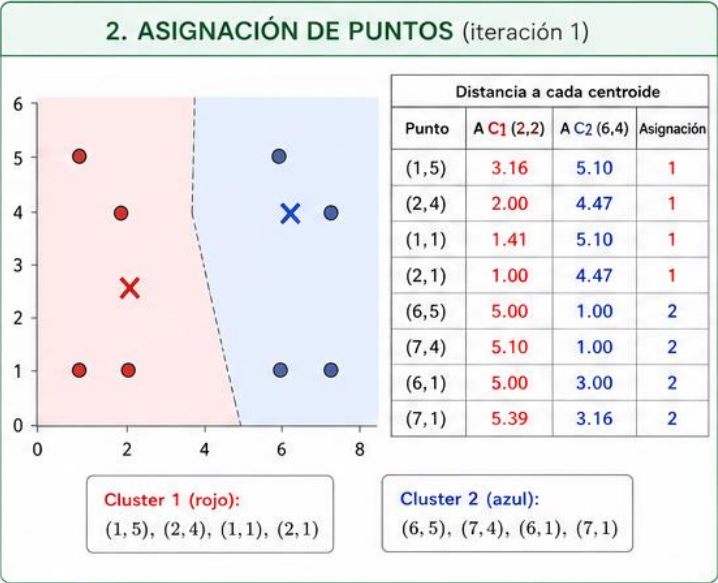
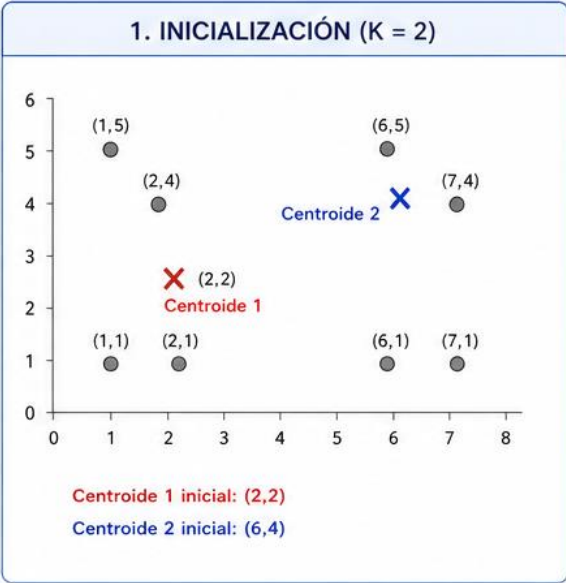
14 **Hasta convergencia**

15 **Devolver** clusters $C_1, \dots, C_K$ y centroides $\mu_1, \dots, \mu_K$

16 **Fin Algoritmo**



4. Kmeans o K-medias



4. Kmeans o K-medias

VENTAJAS



1. Simple y fácil de implementar

El algoritmo es sencillo y fácil de entender.



2. Rápido y escalable

Funciona bien con grandes volúmenes de datos.



3. Convergencia rápida

Normalmente encuentra una solución en pocas iteraciones.



4. Fácil interpretación

Los clusters obtenidos son fáciles de visualizar y entender.



5. Efectivo con clusters compactos

Funciona bien cuando los grupos son esféricos y de tamaño similar.

DESVENTAJAS



1. Requiere definir K previamente

El número de clusters debe elegirse antes de ejecutar el algoritmo.



2. Sensible a outliers

Valores extremos pueden afectar los centroides y los clusters.



3. Sensible a la inicialización

Diferentes centroides iniciales pueden producir resultados distintos.



4. No funciona bien con formas complejas

Tiene dificultad con clusters no esféricos o distribuciones irregulares.



5. Sensible a la escala de los datos

Variables con mayor escala dominan la distancia; requiere normalización.

4. Kmeans o K-medias



KMEANS

Rachit Vargas (MBA | Engineer)

rachit.vargas@ulead.ac.cr

(+506) 8474-3543